

A PROJECT REPORT ON

CAMPUS SAFE

CAMPUS SAFETY & SUPPORT SYSTEM

Python Console Application for Emergency Reporting, Lost & Found Tracking, and Campus Complaint Management.

SUBMITTED BY: DEBRAJ ROY

REG NO: 26BCE10816

COURSE: B. TECH – COMPUTER SCIENCE

YEAR: FIRST YEAR

SUBJECT: PROGRAMMING IN PYTHON

SESSION: 2026-27

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Submitted as a partial fulfillment of his academic coursework.

2. INTRODUCTION

Campus Safe is a simple text-based application realized in Python for managing student security and campus facility complaints in one terminal window. Students may frequently lose items, notice utility outages, and face emergencies on the college premises. Campus Safe provides an easy menu for them to report and log all such complaints live.

3. PROBLEM STATEMENT

Across colleges, reporting of lost belongings and facilities is often done in paper register logs or via staff offices. This causes a number of pain points:

- **Lost Belongings:** No easy way to check if someone picked up their lost items.
- **Emergency:** Informing security guards takes time if there is no centralized desk.
- **Unsorted Complaints:** Urgent fixable facility problems like an open circuit board or leaking pipe are mixed together with other minor complaints.
- **Solution:** One terminal-based application which can receive, store, prioritize and search through all campus safety records on the go.

4. FUNCTIONAL REQUIREMENTS

1. Emergency Reporting

Collects user name, emergency type, location, and description, then instantaneously saves it.

2. Lost & Found Tracking

Enables users to register lost items or report found belongings on campus.

3. Priority Complaint System

Automatically classifies each complaint as HIGH, MEDIUM, or NORMAL depending on the category entered.

4. Search & Display

Provides keyword searching for lost items and displays summary reports of all entries.

5. NON-FUNCTIONAL REQUIREMENTS

- **Ease of Use (Usability):** Simple text menu requires no technical experience to operate.
- **Response Time (Performance):** Instant data storing and retrieval is possible as all operations take place in system RAM.
- **Reliability:** Uses a continuous main program loop so that program remains open until the user explicitly selects to exit.

6. SYSTEM ARCHITECTURE

The program follows a modular functional organizational structure. The main control function “main_()” runs an infinite loop displaying a menu, receiving input, and directing control to individual sub-functions.

7. DESIGN DIAGRAMS

A. Use Case Diagram

Report Emergency, Submit Complaint, Search Lost Item, View All Reports.

B. Workflow Diagram

Start → Display Menu → Get Choice

If 1–6 → Run Function → Loop Menu

If 7 → Print Goodbye → End

C. Sequence Diagram

User select [4] → main_() → main_() calls submission_complaint() → Enter Inputs → Append List → Success

D. Class / Component Diagram

Since the project uses structured functions rather than classes, components are categorized by role:

Data Arrays: Global 2D lists ("emergency_reported", "items_lost", "items_found", "complaint").

Input Handlers: "reporting_emergency()", "items_lost_report()", "items_found_report()", "submission_complaint()".

Display & Search: "viewing_all_reports()", "searching_item_lost()".

E. ER Diagram (Data Storage Schema)

List Name → Attributes (Fields)

emergency_reported → Name, Emergency Type, Location, Description

items_lost → Name, Item Name, Last Seen Location, Description

items_found → Name, Item Name, Found Location, Description

complaint → Name, Category, Location, Details, Priority

8. DESIGN DECISIONS & RATIONALE

2D Lists over Database: Built using Python nested lists due to their simplicity for first-year students and no need for additional setup.

Automated Priority Logic: Complaints checking for categories such as "security" or "electricity" automatically get assigned a HIGH priority tag.

9. IMPLEMENTATION DETAILS

The system is written in normal Python 3. It employs four global empty lists at the beginning. Input functions prompt users via "input()" calls, generate sub-lists, and append them to the main array. Search functionality utilizes case-insensitive string checking via Python's "in" operator and ".lower()" method.

10. SCREENSHOTS / RESULTS

The following screenshots show the working of the Campus Safe Python console application, including emergency reporting, lost and found reporting, complaint submission, search, and the complete report summary.

```
Campus Safe – Python Console

CAMPUS SAFE
Campus Safety & Support System

=====
1. Report Emergency
2. Report Lost Item
3. Report Found Item
4. Submit Complaint
5. Search Lost Item
6. View All Reports
7. Exit
=====
Enter your choice: 1

--- REPORT EMERGENCY ---
Enter your name: Debraj
Enter emergency type (Medical/Fire/Security/Other): Security
Enter location: Main Gate
Describe the emergency: Security assistance required

Emergency reported successfully!
```

Screenshot 1 – Main menu and emergency reporting

```
Campus Safe – Python Console

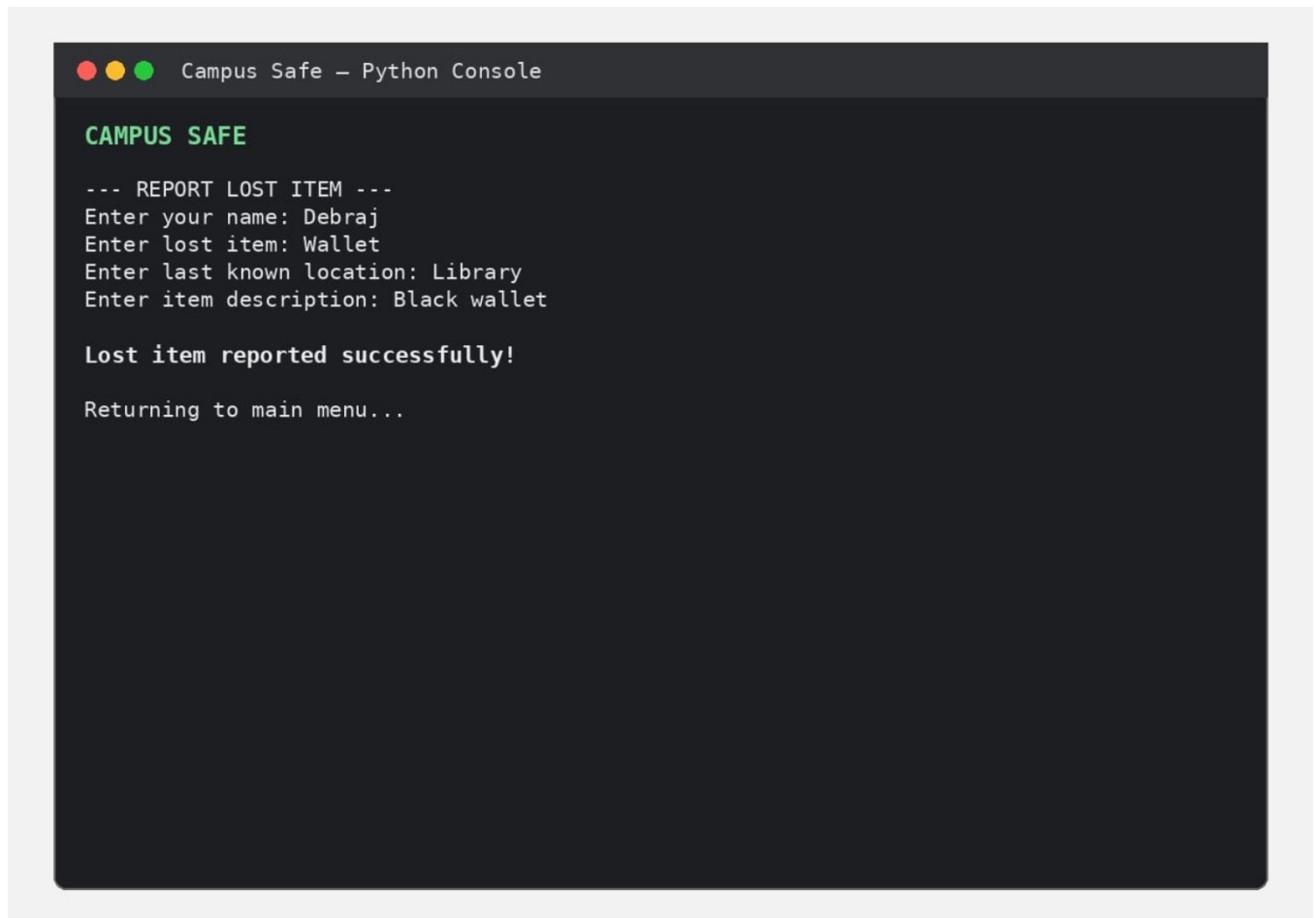
CAMPUS SAFE

--- REPORT EMERGENCY ---
Enter your name: Debraj
Enter emergency type (Medical/Fire/Security/Other): Security
Enter location: Main Gate
Describe the emergency: Security assistance required

Emergency reported successfully!

Returning to main menu...
```

Screenshot 2 – Emergency report successfully submitted



Screenshot 3 – Lost item reporting

CAMPUS SAFE

--- REPORT FOUND ITEM ---

Enter your name: Debraj

Enter found item: Water Bottle

Enter location where it was found: Classroom 204

Enter item description: Blue bottle

Found item reported successfully!

Returning to main menu...

Screenshot 4 – Found item reporting

CAMPUS SAFE

--- SUBMIT CAMPUS COMPLAINT ---

Enter your name: Debraj

Enter category (Security/Cleaning/Electricity/Water/Internet/Other): Security

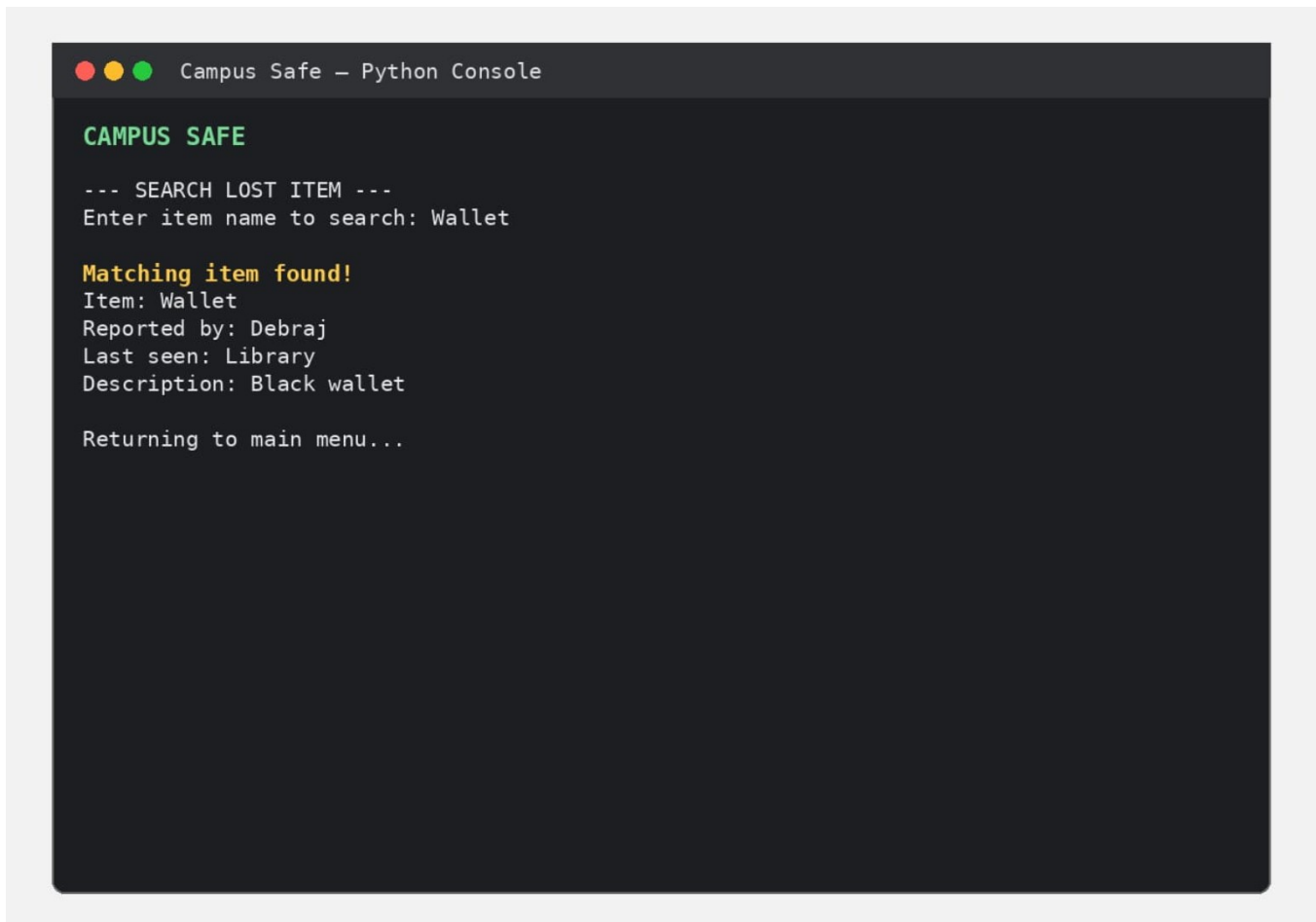
Enter location: Main Gate

Enter your complaint: Gate light is not working

Complaint submitted successfully!

Priority: HIGH

Returning to main menu...



Screenshot 6 – Lost item search result

11. TESTING APPROACH

Test Case	Input Provided	Expected Result	Pass/Fail
Menu Option 1	Choice "1", Enter details	Emergency added to list	PASS
Priority Check	Category "electricity"	Priority printed as "HIGH"	PASS
Item Search	Search keyword "phone"	Matches "iPhone" in list	PASS
Invalid Choice	Choice "9"	Prints "Invalid choice..."	PASS

12. CHALLENGES FACED & FIXED BUGS

Bug 1: Incorrect List Name Variable (NameError)

In "viewing_emergencies()", the loop used "range(len(emergency_reports))", but the list was defined as "emergency_reported". This caused the program to crash.

Fix: Corrected the variable name inside the loop.

Bug 2: Variable Overwriting in Complaints

In "submission_complaint()", variable "c" was used to store category and then immediately reused for complaint details. This erased the category before checking priority.

Fix: Created a separate variable "cat" for category input.

13. LEARNINGS & KEY TAKEAWAYS

- Learned building of multi-option menu loops using "while True" and conditional "if-elif-else" branches.

- Understood how to store structured records via nested 2D lists in Python.
- Gained practice debugging errors like “NameError” and variable shadowing during testing.

14. FUTURE ENHANCEMENTS

- **File Storage:** Save logs to a local file (e.g., “reports.txt”) so that entries are retained after program closure.
- **Graphical Interface (GUI):** Build a visual window with buttons using Python’s “tkinter” library.
- **Admin Password:** Implement a security PIN to ensure only authorized staff can clear or delete old reports.

15. REFERENCES

1. Python Software Foundation (2026). Python 3 Control Flow Tools and List Documentation. Available at: <https://docs.python.org/3/>
2. GeeksforGeeks (2026). Python Data Structures & List Methods Tutorial.